

P.G.D.A.V. College (M), University of Delhi



B. Sc. (H) Computer Science (III Year) SEMESTER V

Software Engineering Project

PATIENT HEALTH MANAGEMENT SYSTEM

Developers:

AISHWARYA PATEL (22053570024)

YASHIKA (22053570042)

Submitted to:

DR. APARNA DATT

CERTIFICATE

This is to certify that the project entitled, “**PATIENT HEALTH MANAGEMENT SYSTEM**” has been done by: **YASHIKA** and **AISHWARYA PATEL** of Bachelor of Science in Computer Science during semester V from PGDAV College (M) University of Delhi.

It embodies the work done by them under the due supervision of **DR. APARNA DATT**.

DECLARATION

I hereby declare that this Project Report titled “**PATIENT HEALTH MANAGEMENT SYSTEM**” submitted to the Department of Computer Science, **PGDAV College** (University Of Delhi) is a record of original work done by the team under the guidance of Dr. Aparna Datt. The information and data given in the report is authentic to the best of the team knowledge. This Project Report is not submitted to any other university or institution for the award of any degree, diploma or fellowship or published any time before.

ACKNOWLEDGEMENT

The contentment that is achieved on the successful completion of any task is incomplete without mentioning the names of the people who made it possible with their consistent guidance, support and indispensable encouragement. The Project was jointly undertaken by YASHIKA and AISHWARYA PATEL as their V semester Software Engineering Project, under the able guidance and supervision of DR. APARNA DATT. Our primary thanks to her, who poured over every inch of our project with painstaking attention and helped us throughout the working of the project. It's our privilege to acknowledge our deepest sense of gratitude to her for her inspiration which has helped us immensely. We are extremely grateful to her for unstilted support and encouragement in the preparation of this project.

INDEX

<u>S.NO.</u>	<u>TOPIC</u>	<u>PAGE NO.</u>
1	Problem Statement	6
2	Evolutionary Process Model	7
3	SRS	8-12
4	Users and their Characteristics	13
5	Use Case	14-17
6	DFD	18-20
7	ERD	21-22
8	Database Used In Project and Queries	23-25
9	Timeline Chart	26
10	UI Implementation	27
11	Pseudocode	28
12	Cyclomatic Complexity	29
13	Risk Management	30
14	Testing	31-32
15	Conclusion	33
16	Bibliography	34

PROBLEM STATEMENT

The current patient health management systems are fragmented, leading to challenges in accessing accurate patient record, tracking health progress, managing appointments, and ensuring timely communication. Due to this healthcare professional's struggle with incomplete information and time-consuming administration.

To address these challenges we propose development of centralized patient healthcare management that includes **patient's record** and **appointment schedule** to allow tracking **patient's health record** and **timely diagnosis**.

EVOLUTIONARY PROCESS MODEL

The evolutionary model is a combination of both **iterative** and **incremental** models. The main aim of this evolutionary model is to deliver the products in total processes over time.

Based on the evolutionary model, we can **divide** the development **model into many modules** which help the developer build and transfer incrementally. We can also develop the skeleton of the initial product. Also, it refines the project to increase levels of capability by adding new functionalities in successive versions.

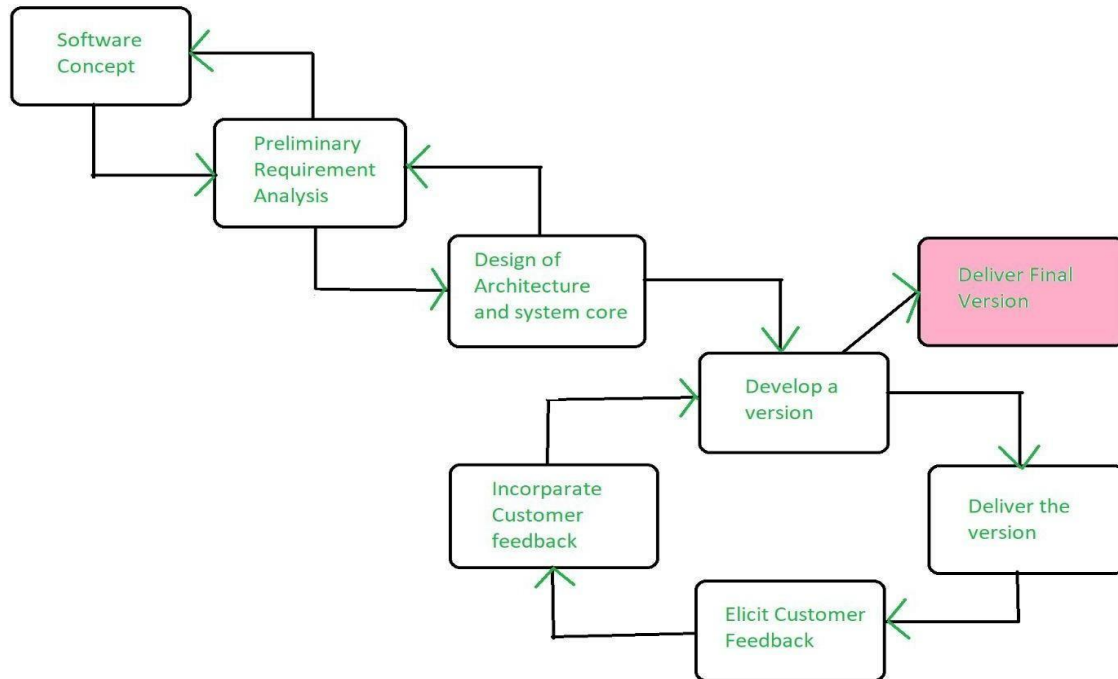


Fig 1: Evolutionary Process Model

Reason for selecting this model:

An evolutionary model in patient health management project will help the product to manage according to need of every patient will help to make changes after every iteration . It will also help the model to adapt according to patient's treatment plan. This ensures the care is tailored according to each patient's specific need.

SOFTWARE REQUIREMENT SPECIFICATION (SRS)

1. Introduction

- **Purpose**

The purpose of this document is to define the requirements for a Patient Management System (PMS). The system aims to centralize patient health records, manage appointments, track patient progress, and facilitate communication between patients, doctors, and receptionists. This SRS will outline the functional and non-functional requirements of the system.

- **Scope**

The Patient Management System will serve as a comprehensive platform for:

- Storing and updating patient medical records.
- Scheduling and managing appointments.
- Billing and invoice management.
- Enabling doctors to track and update patient health progress.
- Providing receptionists with the ability to manage patient information and appointments.

The users of the system include:

- **Patients:** Can register, book/cancel appointments, and view medical records.
- **Doctors:** Can view patient records, manage appointments, update medical history, and prescribe medication.
- **Receptionists:** Manage appointments, validate patient information, and handle billing processes.

- **Definitions, Acronyms, and Abbreviations**

- **PMS:** Patient Management System
- **SRS:** Software Requirements Specification
- **DFD:** Data Flow Diagram
- **EHR:** Electronic Health Record

- **Overview**

This document provides an in-depth description of the **PMS**. **Section 2** discusses the overall system requirements, **Section 3** details functional requirements, and **Section 4** describes non-functional requirements.

2. Overall Description

- **Product Perspective**

The **PMS** will be a web-based application accessible by patients, doctors, and receptionists. It will be hosted on a secure server and available 24/7 to authorized users. The system will centralize medical records, scheduling, and billing, which will be accessible by all authorized users based on their roles.

- **Product Features**

- **Patient Registration:** Patients can register and update their profiles.
- **Appointment Scheduling:** Patients can book, view, or cancel appointments with doctors.
- **Patient Health Records:** Doctors can update and view medical records, and patients can view their own medical history.
- **Billing Management:** Generates invoices for patient visits and medical services.
- **Prescription Management:** Doctors can add and update prescriptions for patients.
- **Role-Based Access:** Different levels of access are provided for patients, doctors, and receptionists.

- **User Classes and Characteristics**

- **Patients:** Non-technical users who will interact with the system to register, manage their profile, view doctor's profile, book appointments, view prescriptions, and access medical records.
- **Doctors:** Medical professionals who will use the system to view patient records, update medical history, issue prescriptions, manage appointments and manage their own profile.
- **Receptionists:** Administrative users responsible for managing appointments, validating patient information, and handling billing.

- **Operating Environment**

- Web-based system accessible on modern browsers (Chrome, Firefox, Safari, Edge).
- Hosted on a cloud platform for 24/7 availability.
- Database: Relational database (e.g., MySQL or PostgreSQL).
- Secure HTTPS protocol for data protection.

- **Design and Implementation Constraints**

- Compliance with **HIPAA** (Health Insurance Portability and Accountability Act) for patient data security.
- Integration with existing hospital management systems (if applicable).
- Responsive design for access on various devices (desktops, tablets, smartphones).

3. Functional Requirements

- **Patient Registration**

- The system shall allow patients to register by providing personal details (name, contact information, medical history).
- The system shall validate patient data during registration to avoid duplicates.
- The system shall send a confirmation email after successful registration.

- **Appointment Scheduling**

- The system shall allow patients to schedule appointments with doctors based on availability.
- The system shall allow patients to cancel or reschedule appointments.
- The system shall send a notification to the patient and doctor upon successful appointment scheduling, rescheduling, or cancellation.

- **Patient Health Records**

- The system shall allow doctors to access patient medical records and update them after consultations.
- The system shall allow patients to view their health records, including past diagnoses and treatments.
- The system shall securely store and protect patient health records from unauthorized access.

- **Prescription Management**

- The system shall allow doctors to add, view, and update patient prescriptions.
- The system shall allow patients to view their prescribed medication.

- **Billing and Invoice Management**

- The system shall generate bills after appointments based on services provided.
- The system shall allow patients to view and pay invoices through the system.
- The system shall allow receptionists to validate the billing process and correct any errors.

- **User Roles and Permissions**

- **Patients** shall have access to register, schedule/cancel appointments, and view their health and billing information.
- **Doctors** shall have access to view and update patient records, view their appointments, and manage prescriptions.
- **Receptionists** shall have access to view and manage appointments, validate patient information, and handle billing.

4. Non-Functional Requirements

- **Performance Requirements**

- The system should respond to user requests (e.g., patient registration, appointment scheduling) within 3 seconds.
- The system should support up to 500 concurrent users.

- **Security Requirements**

- The system must comply with **HIPAA** standards to protect patient information.
- All sensitive data must be encrypted both in transit (using HTTPS) and at rest.
- User authentication shall be implemented using secure login methods (e.g., 2FA).

- **Usability Requirements**

- The system should be intuitive and easy to use for non-technical users (patients).
- The system should be accessible for users with disabilities, adhering to **WCAG 2.0** guidelines.

- **Reliability and Availability**

- The system shall be available 99.9% of the time, with planned downtime for maintenance.
- Automated backups should occur daily to ensure data recovery in case of failure.

- **Maintainability**

- The system shall be modular, making it easy to maintain and upgrade.
- The system should allow for the addition of new features with minimal disruption.

- **Portability**

- The system should be accessible across all modern browsers and devices (desktop, tablet, smartphone).

5. System Models

- **Data Flow Diagrams (DFD):** Illustrate the flow of information between patients, doctors, receptionists, and key system processes (patient registration, appointments, billing, etc.).
- **Entity-Relationship Diagram (ERD):** Shows the relationships between entities such as Patients, Doctors, Appointments, and Billing Records.

USERS AND THEIR CHARACTERISTICS

The users of this project includes hospitals, doctors and patients.

The users have different access according to their needs and level of privilege.

DOCTORS:

Doctors will have access to the interface and will be able to get patient details, appointment details and their own details.

ADMINISTRATOR:

Administrator's from hospitals will have access to the interface and will be able to view patient details and generate their bills.

PATIENTS:

These are the people who are making an appointment with the doctor through this interface. They will be able to get doctor details, bills and provide their own details for further diagnosis.

USE CASE

A use case is a methodology used in system analysis to identify, clarify and organize system requirements. The use case is made up of a set of possible sequences of interactions between systems and users in a particular environment and related to a particular goal. The method creates a document that describes all the steps taken by a user to complete an activity.

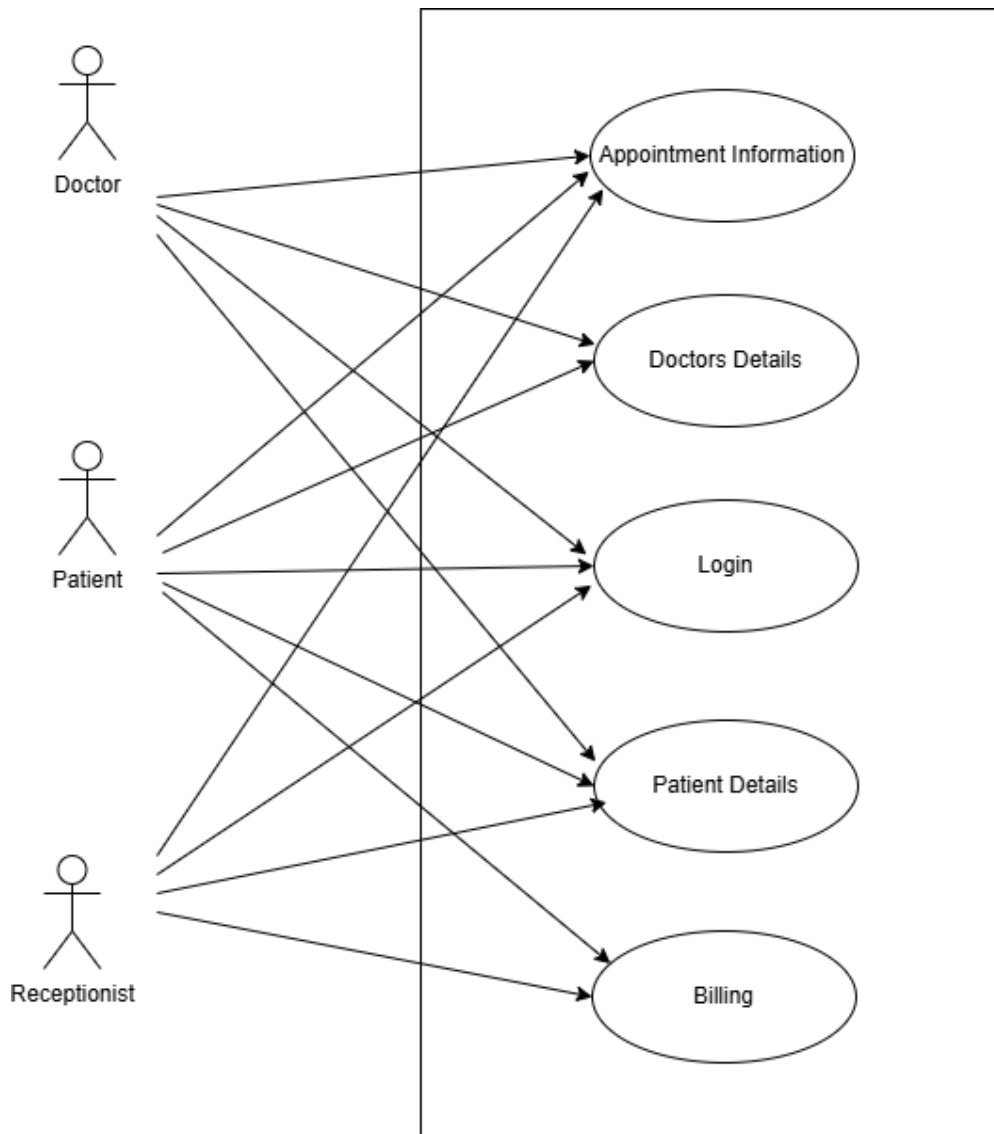


Fig 2: Use Case Diagram

1. Introduction: (Patient)

This use case describes how a patient can take appointments in the patient health management system. It involves viewing appointment slots, booking, rescheduling, or cancelling an appointment with a doctor.

2. Actors:

- **Primary Actor:** Patient

3. Pre-conditions:

- The patient must be registered in the system.
- The doctor's schedule must be available in the system.

4. Post-conditions:

- **Success:** The appointment is booked, rescheduled, or cancelled, and the system updates the doctor's and patient's schedules.
- **Failure:** If no slots are available, an appropriate message is displayed.

5. Flow of Events:

5.1 Basic Flow:

- The patient logs into the system.
- The patient selects "Take Appointment."
- The patient selects a doctor and an available time and date slot.
- The system confirms the booking and updates the schedules.

5.2 Alternative Flows:

If the selected slot is no longer available:

- The system notifies the patient and suggests alternative slots.

If the patient wishes to reschedule:

- The patient selects a new slot, and the system updates the previous booking.

6. Special Requirements:

The system should allow real-time slot availability checks.

Notifications should be sent to the patient upon successful booking or changes.

1. Introduction (Doctor)

Doctors access their schedules to view upcoming appointments, check patient details, and prepare for consultations.

2. Actors

Primary: Doctor

3. Pre-Conditions:

- The doctor is registered and authorized to access the system.
- The appointment schedule is updated.

4. Post-Conditions:

- The doctor successfully views the schedule and patient details.
- The doctor is prepared for the appointment.

5. Flow of Events:

5.1 Basic Flow:

- The doctor logs into the system.
- The doctor navigates to the schedule interface.
- The system displays a list of appointments with associated patient details.

5.2 Alternative Flows:

If the doctor notices a scheduling error, they notify the receptionist to correct it.

6. Special Requirements:

Patient details must remain confidential and accessible only to authorized personnel.

1. Introduction (Receptionist)

The receptionist schedules, updates, and cancels appointments, ensuring the system stays accurate and patients are notified.

2. Actors:

Primary: Receptionist

3. Pre-Conditions:

- The receptionist has access to the system.
- The doctor's and patient's information is in the system.

4. Post-Conditions:

- Appointments are accurately updated in the system.
- Notifications are sent to patients and doctors as necessary.

5. Flow of Events:

5.1 Basic Flow:

- The receptionist logs into the system.
- The receptionist views available slots and books or updates an appointment for a patient.
- The system updates the database and sends notifications to involved parties.

:

5.2 Alternative Flows:

If the patient requests rescheduling, the receptionist repeats the process with the new date.

6. Special Requirements:

Ensure accuracy in patient and doctor details to avoid errors.

Data Flow Diagram(DFD)

A data flow diagram or bubble chart is a graphical representation of the flow of data through a system. It clarifies system requirements and identify major transformations.

The objective of the DFD is to represent a system at different level of abstraction.

DFD(level 0):

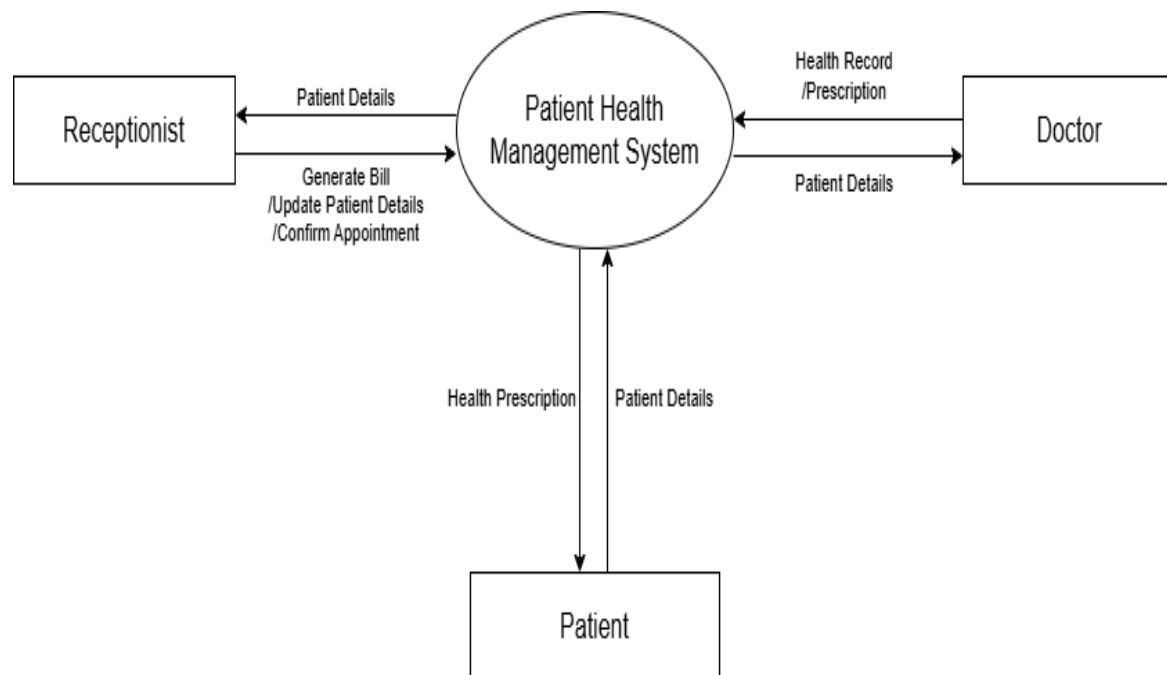


Fig 3: DFD Level 0

DFD(Level 1):

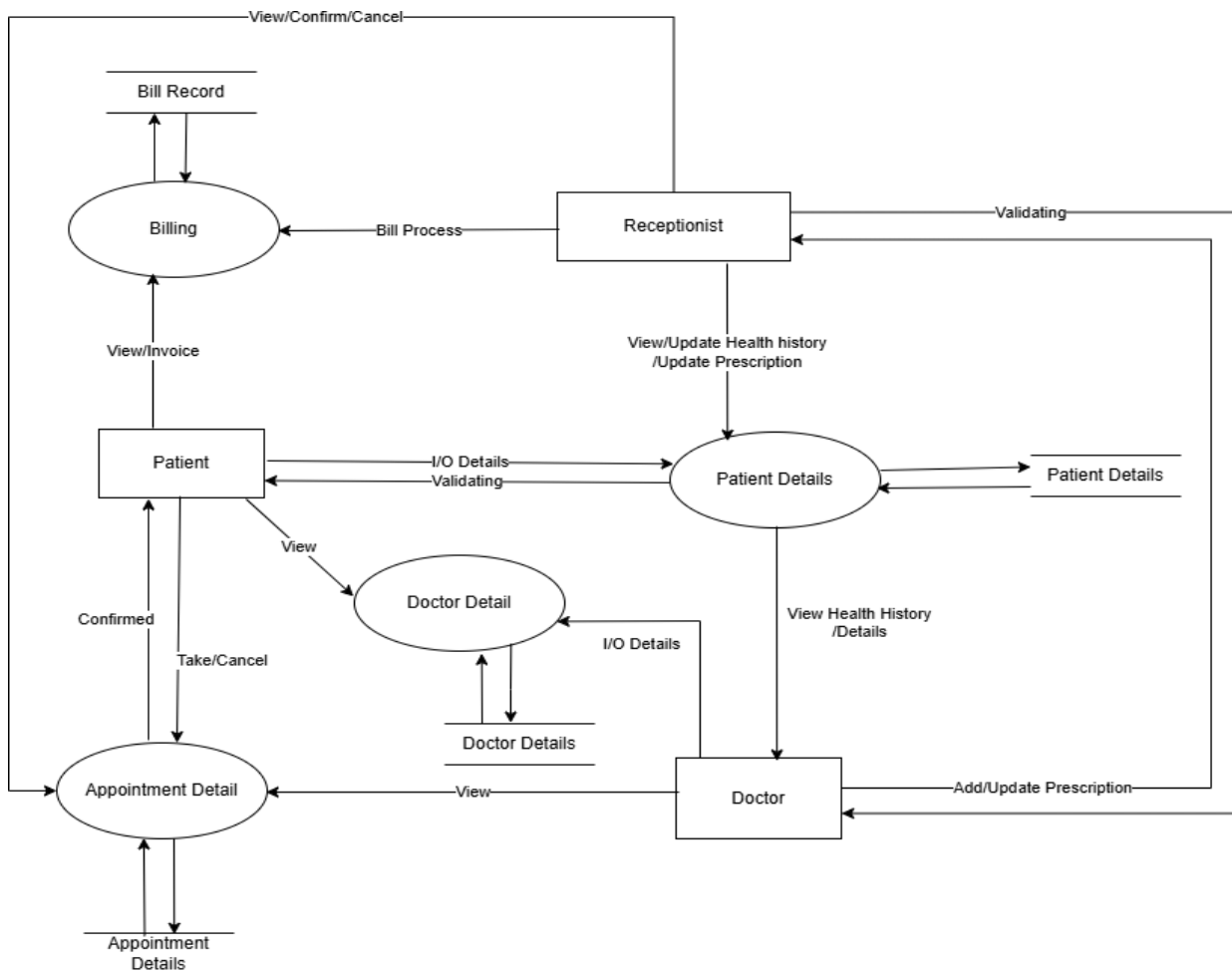


Fig 4: DFD Level 1

DFD(level 2):

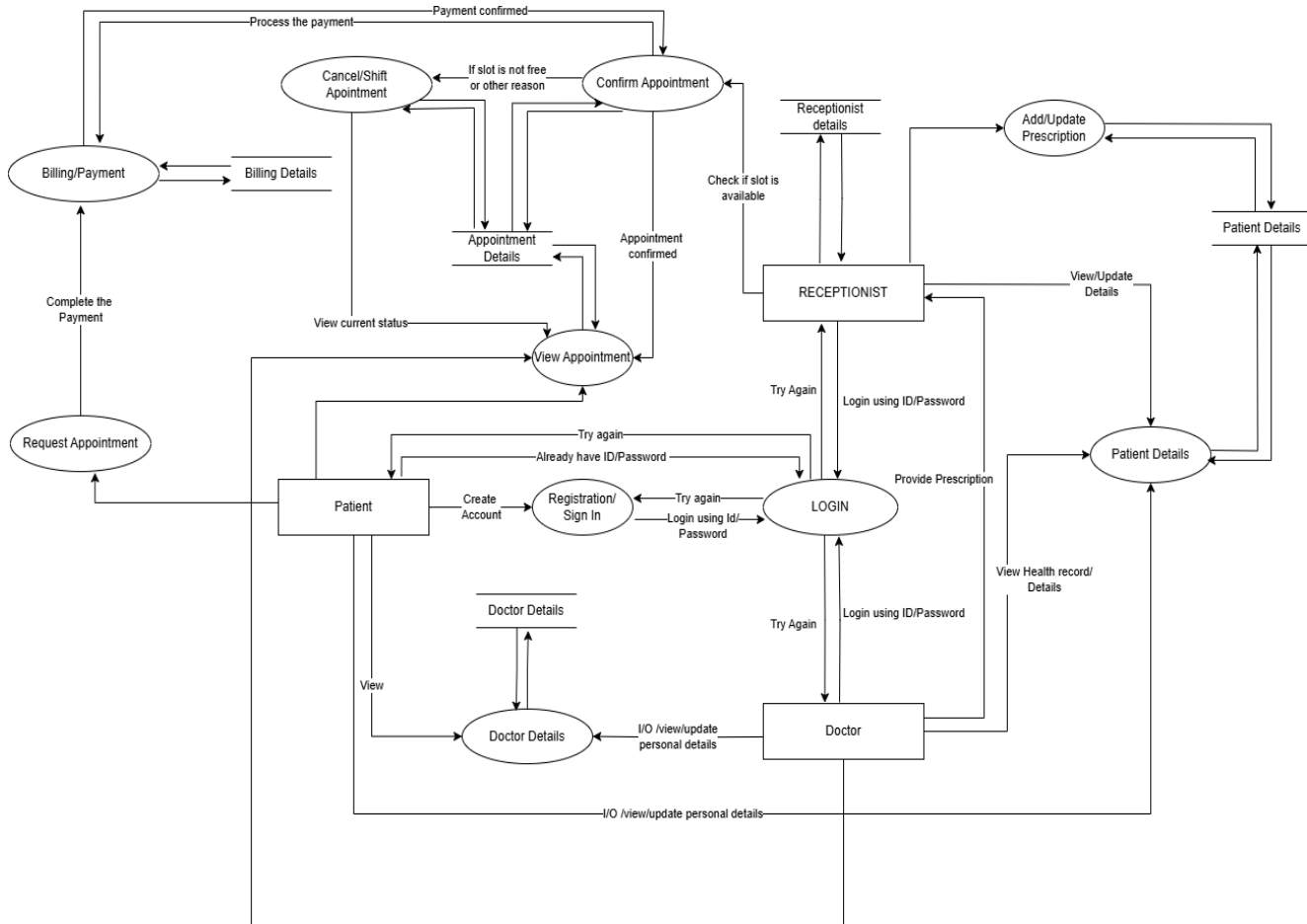


Fig 5: DFD Level 2

Entity Relationship Diagram(ERD)

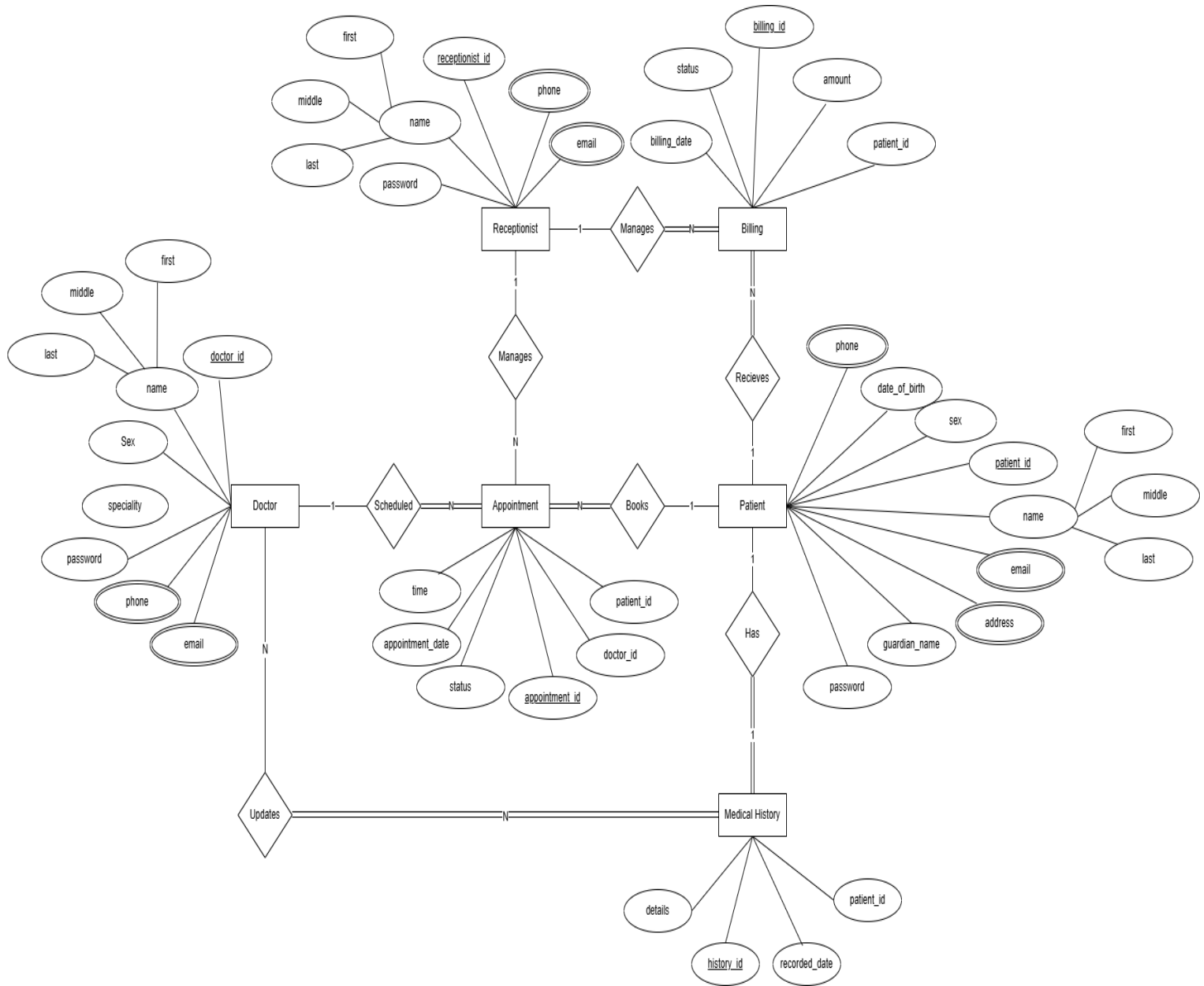


Fig 6: ERD

References For ERD

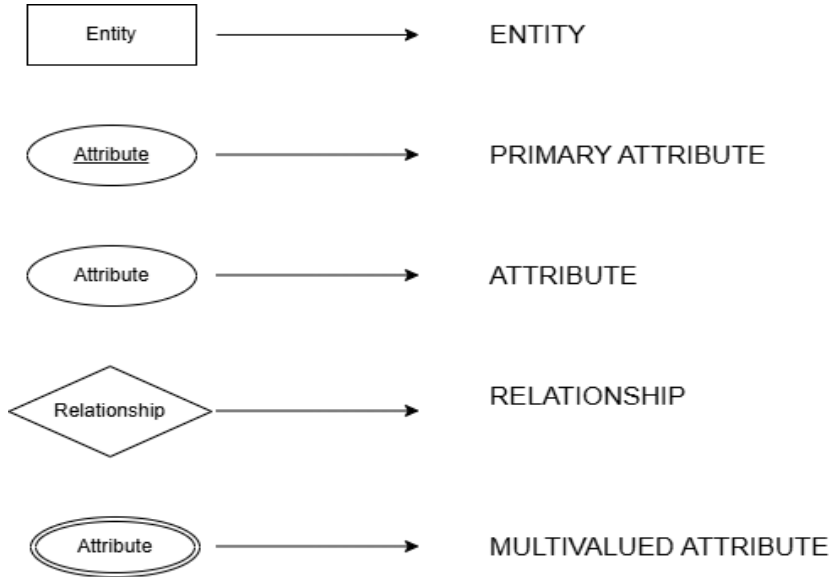


Fig 7: ERD References

Mapping Of ERD

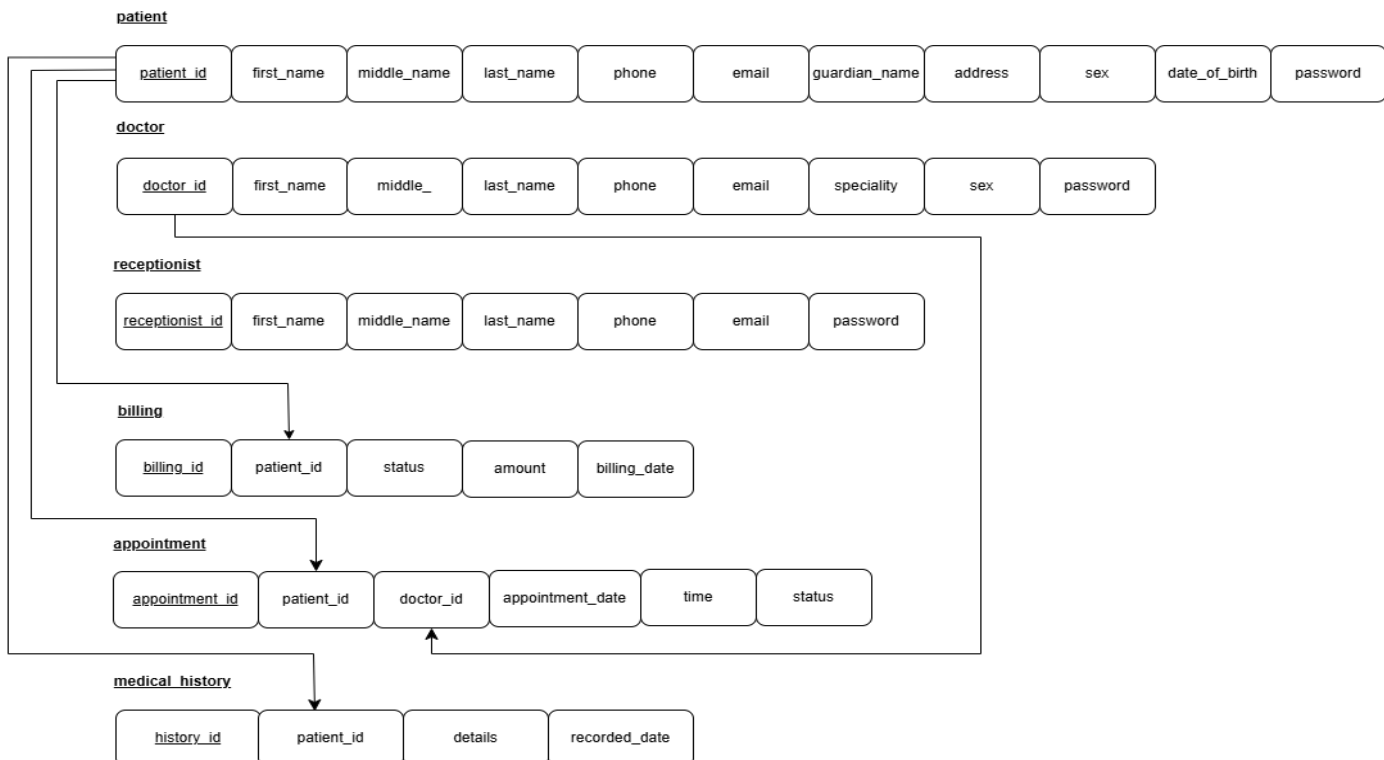


Fig 8: ERD Table

Databases Used In The Project

Fig 9: Database for the project

Patient

#	Name	Type	Collation	Attributes	Null	Default
1	patient_id 🔑	int(11)			No	None
2	first_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
3	middle_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
4	last_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
5	phone	varchar(10)	utf8mb4_unicode_ci		No	None
6	guardian_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
7	address	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
8	sex	varchar(255)	utf8mb4_unicode_ci		No	None
9	email	varchar(255)	utf8mb4_unicode_ci		No	None
10	dob	date			Yes	NULL
11	password	varchar(40)	utf8mb4_unicode_ci		No	None

Appointment

#	Name	Type	Collation	Attributes	Null	Default
1	appointment_id 🔑	int(11)			No	None
2	patient_id 🔑	int(11)			Yes	NULL
3	doctor_id 🔑	int(11)			Yes	NULL
4	appointment_date	date			Yes	NULL
5	time	time			Yes	NULL
6	status	varchar(225)	utf8mb4_unicode_ci		No	Booked

doctor

#	Name	Type	Collation	Attributes	Null	Default
1	doctor_id 🔑	int(11)			No	None
2	first_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
3	middle_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
4	last_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
5	sex	varchar(255)	utf8mb4_unicode_ci		No	None
6	speciality	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
7	phone	varchar(10)	utf8mb4_unicode_ci		No	None
8	email	varchar(255)	utf8mb4_unicode_ci		No	None
9	password	varchar(40)	utf8mb4_unicode_ci		No	None

receptionist

#	Name	Type	Collation	Attributes	Null	Default
1	receptionist_id 🔑	int(11)			No	None
2	first_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
3	middle_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
4	last_name	varchar(255)	utf8mb4_unicode_ci		Yes	NULL
5	phone	varchar(10)	utf8mb4_unicode_ci		No	None
6	email	varchar(255)	utf8mb4_unicode_ci		No	None
7	password	varchar(40)	utf8mb4_unicode_ci		No	None

medical_history

#	Name	Type	Collation	Attributes	Null	Default
1	history_id 🔑	int(11)			No	None
2	patient_id 🔑	int(11)			Yes	NULL
3	recorded_date	date			Yes	NULL
4	details	varchar(500)	utf8mb4_unicode_ci		Yes	NULL

billing

#	Name	Type	Collation	Attributes	Null	Default
1	billing_id 🔑	int(11)			No	None
2	patient_id 🔑	int(11)			Yes	NULL
3	billing_date	date			Yes	NULL
4	amount	int(11)			Yes	NULL
5	status	varchar(225)	utf8mb4_unicode_ci		Yes	NULL

NOTATION: 🔑 Primary

Queries

Fig 10: Queries

Select * from patient;

Output-

patient_id	first_name	middle_name	last_name	phone	guardian_name	address	sex	email	dob	password
1	Aman	Pratap	Singh	907845167	Ram Singh	Lajpat Nagar, Delhi-110024	Male	aman@gmail.com	1992-09-14	
2	Rishita		Singh	145672898	Priyansh Singh	Prayagraj, Uttar Pradesh-100003	Female	rishi123@gmail.com	1989-10-01	
3	Simran		Pal	145672898	Mahesh Pal	Jaipur, Rajasthan-100028	Female	mahesh5@gmail.com	1997-01-21	
4	Vijay		Prakash	2857639012	Arun Prakash	Kolkata, West Bengal-100007	Male	vijay2@gmail.com	1982-07-09	
5	Ankit	Kumar		6789021222	Arjit Kumar	Amritsar, Punjab-100012	Male	Ankit09@gmail.com	2000-05-11	
7	John	A	Doe	1234567898	NULL	NULL		johndoe@example	NULL	\$2y\$10\$8UqnTa1CRhI24WwENH/
17	Raj			1111111111	NULL	NULL		raj@gmail.com	2024-11-13	raj

Select * from doctor;

Output-

doctor_id	first_name	middle_name	last_name	sex	speciality	phone	email	password
11	Koushiki		Patel	Female	Oncologist	1000239857	koushikipatel5@gmail.com	
12	Atharv		Vats	Male	cardiologist	4857629871	Atharv5@gmail.com	
13	Manas		Katiyar	Male	neurologist	5768943988	manask@gmail.com	
14	Kavya		Sharma	Female	Dermatologist	1134544789	kavya@gmail.com	
15	Utpal			Male	Geriatrics	3900457887	utpalp6@gmail.com	

Select * from receptionist;

Output-

receptionist_id	first_name	middle_name	last_name	phone	email	password
21	Archita		Gupta	2899850078	architagupta@gmail.com	
22	Ashutosh		Pandey	1784311254	Ashu@gmail.com	
23	Sanchita	Kumari		1986878886	Sanchitak@gmail.com	
24	Sanjay	Kumar	Yadav	1867556475	Sanjay45@gmail.com	
25	Khushi			1867556475	Khushi11@gmail.com	

Select * from appointment;

Output-

appointment_id	patient_id	doctor_id	appointment_date	time	status
1001	1	12	2024-07-09	13:00:00	canceled
1002	2	11	2024-08-15	10:30:00	Booked
1003	5	13	2024-08-15	12:00:00	Booked
1004	3	14	2024-09-10	14:00:00	Booked
1005	4	15	2024-09-11	11:00:00	Booked
1014	7	11	2024-11-29	12:30:00	booked
1015	1	11	2024-11-23	12:00:00	booked
1016	1	13	2024-11-23	13:00:00	booked

Select * from medical_history;

Output-

history_id	patient_id	recorded_date	details
201	1	2024-07-09	High blood pressure
202	2	2024-08-15	Pancreatic cancer
203	3	2024-09-10	Psoriasis
204	4	2024-09-11	high fever
205	5	2024-08-15	Parkinson Disease

Select * from billing;

Output-

billing_id	patient_id	billing_date	amount	status
3001	1	2024-07-09	2000	Completed
3002	2	2024-08-15	5000	Completed
3003	3	2024-09-10	3000	Pending
3004	4	2024-09-11	3500	Completed
3005	5	2024-08-15	4000	Pending

TIMELINE CHART

When creating a software project schedule, you begin with a set of tasks. The work breakdown is input as a task network. A Time-line chart also known as *Gantt Chart*, is generated, this can be generated for the entire project or for each project function individually.

The Time-line chart of our project is as follow:

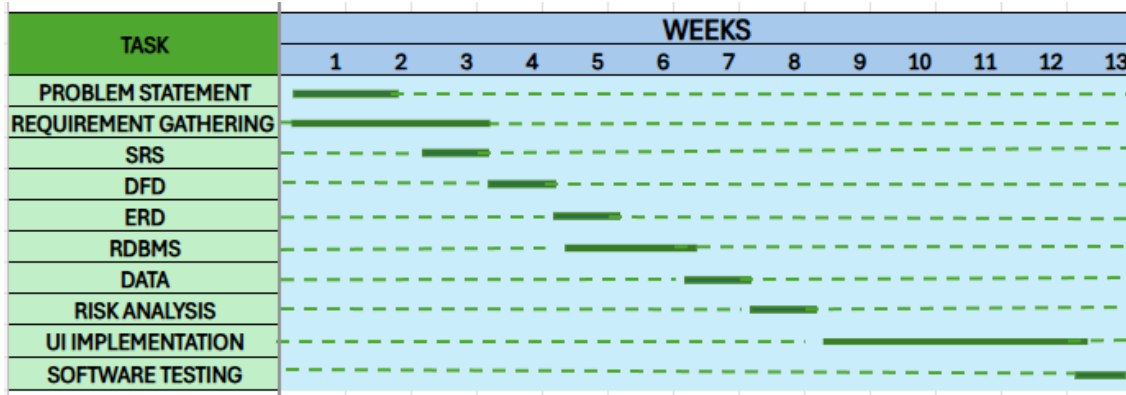
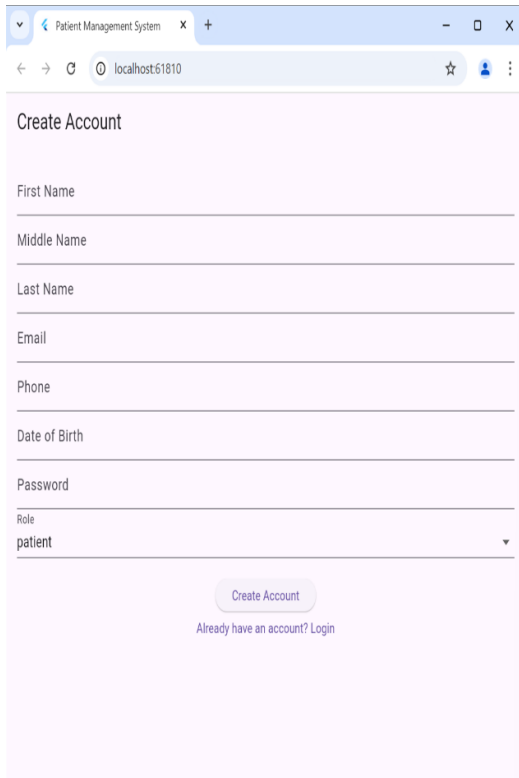


Fig 11: Timeline chart

UI IMPLEMENTATION



Browser: Patient Management System, localhost:61810

Create Account

First Name

Middle Name

Last Name

Email

Phone

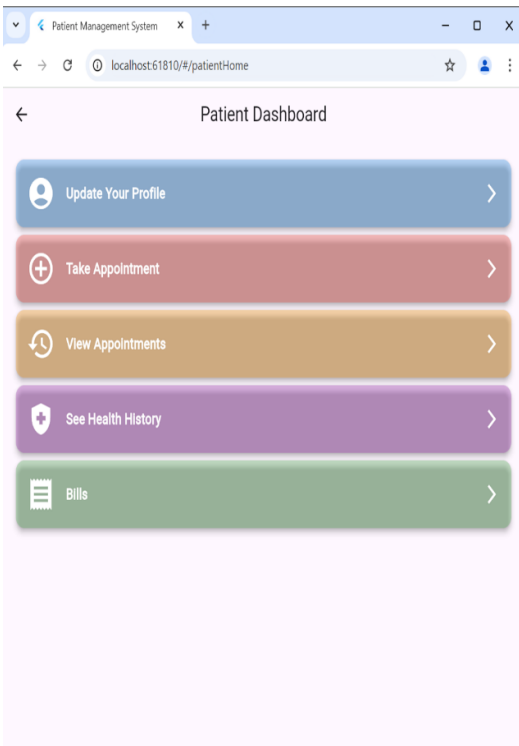
Date of Birth

Password

Role
patient

Create Account

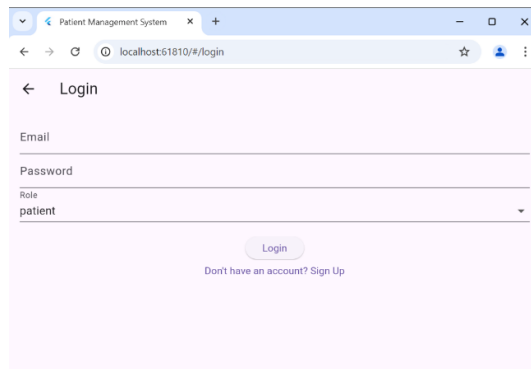
Already have an account? Login



Browser: Patient Management System, localhost:61810/#/patientHome

Patient Dashboard

- Update Your Profile
- Take Appointment
- View Appointments
- See Health History
- Bills



Browser: Patient Management System, localhost:61810/#/login

Login

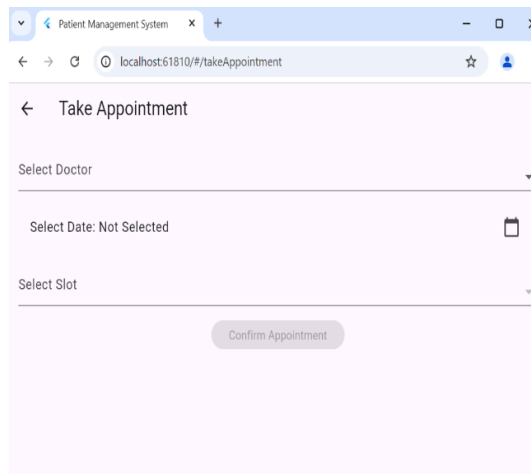
Email

Password

Role
patient

Login

Don't have an account? Sign Up



Browser: Patient Management System, localhost:61810/#/takeAppointment

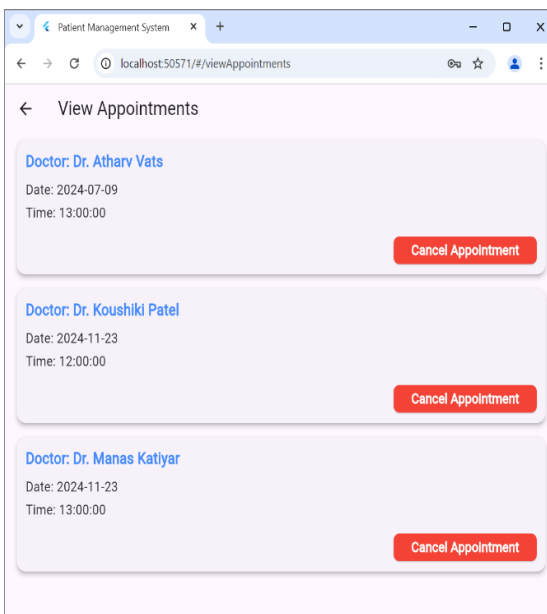
Take Appointment

Select Doctor

Select Date: Not Selected

Select Slot

Confirm Appointment



Browser: Patient Management System, localhost:50571/#/viewAppointments

View Appointments

Doctor: Dr. Athary Vats	Date: 2024-07-09	Time: 13:00:00	Cancel Appointment
Doctor: Dr. Koushiki Patel	Date: 2024-11-23	Time: 12:00:00	Cancel Appointment
Doctor: Dr. Manas Katiyar	Date: 2024-11-23	Time: 13:00:00	Cancel Appointment

PSEUDOCODE

BEGIN CreateAccountPage

DEFINE form variables:

- firstName
- middleName
- lastName
- email
- phone
- password
- role (default: "patient")

FUNCTION createAccount:

BEGIN

SEND HTTP POST request to server with the following data:

- firstName
- middleName
- lastName
- email
- phone
- password
- role

GET response from server

IF response is successful THEN

PRINT "Account created successfully"

ELSE

PRINT "Error creating account"

END FUNCTION

FUNCTION buildUI:

BEGIN

DISPLAY form to the user with the following fields:

- Text input for First Name
- Text input for Middle Name
- Text input for Last Name
- Text input for Email
- Text input for Phone Number
- Text input for Password (hidden text)
- Dropdown for selecting Role ("patient", "doctor", "receptionist")

BUTTON "Create Account":

WHEN clicked:

VALIDATE form fields:

- Ensure no field is empty
- Ensure email format is correct
- Ensure password meets security requirements

IF form is valid THEN

CALL createAccount to submit the form

ELSE

DISPLAY error message ("Please fill in all fields correctly")

BUTTON "Login":

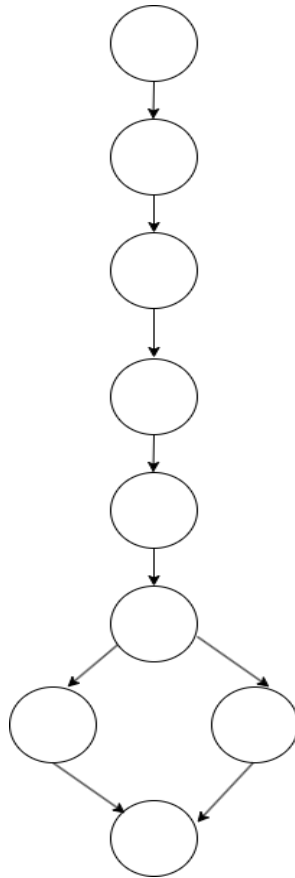
WHEN clicked:

NAVIGATE to the Login page

END FUNCTION

END CreateAccountPage

CYCLOMATIC COMPLEXITY



Complexity =			
$E - N + 2$	OR	$P + 1$	OR Bounded region + 1
$9 - 9 + 2$		$1 + 1$	$1 + 1$
$= 2$		$= 2$	$= 2$

RISK MANAGEMENT

Risk management is the process of identifying, assessing, and mitigating risks that could potentially impact an organization's goals or objectives. It involves analyzing and evaluating potential risks and developing strategies to minimize or avoid them.

Risk management plays a critical role in helping organizations to achieve their objectives, protect their assets, and ensure their long-term sustainability.

Steps followed in risk management are:

- Identifying risks
- Assessing risks
- Mitigating risks
- Monitoring of risks
- Contingency Plans

RISK ID	DESCRIPTION	LIKELIHOOD	IMPACT	MITIGATION PLAN
R001	System Down Due To Server	Medium	High	Implement robust server backup and recovery plan
R002	Patient Data Breach	High	High	Ensure data encryption and implement strict access controls
R003	Errors In Appointment Scheduling Due To Conflicts	Medium	Medium	Add validation rules to check for conflicts
R004	Inaccurate Billing Calculation	Medium	High	Implement automated billing and error-checking
R005	Doctor's Unavailability	Low	Medium	Maintain list of substitute doctors for emergency

Fig 17: Risk management sheet

TESTING

Testing is a process of executing a program with the aim of finding error. To make our software perform well it should be error free. If testing is done successfully, it will remove all the errors from the software.

TEST ID:001	Patient Registration
PURPOSE	To validate the process of registering a new patient in the system.
PRECONDITION	Database connection established, registration page loaded.
INPUTS	Patient name, Password
EXPECTED OUTCOMES	A unique patient_id is generated, and the details are stored in the database.
POST CONDITION	Patient data is available for further appointments or inquiries.

Fig 12: Test case 1

TEST ID:002	Doctor's Appointment
PURPOSE	To verify the assignment of a doctor to a patient's appointment.
PRECONDITION	Patient and doctor records already exist in the system.
INPUTS	Appointment details including patient_id, doctor_id, and date.
EXPECTED OUTCOMES	An appointment_id is generated, and the details are stored in the database.
POST CONDITION	The appointment is available for viewing and updating.

Fig 13: Test case 2

TEST ID:003	Creation and Updates
PURPOSE	To ensure an appointment can be created and updated successfully.
PRECONDITION	Doctor and patient details are available.
INPUTS	Appointment details, doctor_id, patient_id.
EXPECTED OUTCOMES	Appointment details saved or updated.
POST CONDITION	Updated data reflects in all related reports.

Fig 14: Test case 3

TEST ID:004	Patient medical history access
PURPOSE	To validate access to a patient's medical history.
PRECONDITION	Patient has previous medical records.
INPUTS	patient_id.
EXPECTED OUTCOMES	Display of all medical records for the selected patient.
POST CONDITION	Data should remain secure and unaltered.

Fig 15: Test case 4

TEST ID:005	Billing process
PURPOSE	To test the generation of a bill for a patient's visit.
PRECONDITION	Appointment and treatment details exist.
INPUTS	patient_id, treatment details, and consultation fees.
EXPECTED OUTCOMES	A bill is generated with a unique billing_id.
POST CONDITION	Bill details are stored for future reference.

Fig 16: Test case 5

CONCLUSION

In conclusion, the proposed centralized patient healthcare management system aims to overcome the fragmentation and inefficiencies currently present in healthcare systems. By integrating patient records, appointment schedules, and health tracking into a single platform, this solution will enhance the accessibility and accuracy of patient information, streamline administrative tasks, and enable healthcare professionals to deliver timely and informed care. Ultimately, the system will improve both the patient experience and healthcare outcomes by facilitating more efficient communication, better management of appointments, and more effective health progress monitoring.

BIBLIOGRAPHY

1. Pressman, R.S., & Maxim, B.R. (2020). Software Engineering:A Practitioner's Approach. 9th ed. McGraw-Hill
2. Aggarwal, K. K., & Singh, Y. (2007). Software Engineering. 3rd ed. New Age International Publishers.
3. Jalote, P. An Integrated Approach to Software Engineering, 3rd ed., Narosa Publishing House, 2005.